

Predicting Employee Promotion State

James Finlinson, Ayden Swain, Hoon Lee, Yukina Soga

CS 270, Fall 2024

Department of Computer Science
Brigham Young University

Abstract

To assist employers in making informed promotion decisions, we have developed a machine learning model to predict an employee's role within a company. Utilizing a dataset comprising employee information from 706 restaurants, we trained various models to determine the most effective predictor of employee roles. Our most accurate model was the K-Nearest Neighbor, which achieved an accuracy of 93.08%. KNNs significant outperformance of the other models stems from its ability to account for locality within individual businesses.

1 Introduction

Employee promotion in corporate organizations is a key factor that influences employee motivation. However, predicting when each employee will be promoted is a challenging task. With advances in data analytics, many companies now track employee performance to optimize their processes. This inspired us to explore whether employee data could be used to predict promotion timelines. In our project, we aim to predict when employees are likely to be promoted using machine learning techniques. We utilize private data from over 150,000 restaurant employees, including critical factors such as job evaluations, changes in job positions, and employment tenure. Furthermore, we strive to identify employees who meet promotion criteria but have yet to be evaluated. This paper evaluates the effectiveness of various machine learning models for predicting promotions and explores whether these models can offer reliable insights for decision-making.

2 Methods

2.1 Data extraction

We secured access to a private firestore database containing employee information for 706 restaurants. This database allowed us to submit queries to retrieve the specific data we needed. However, since the database was

not originally designed to support machine learning models, we extracted and transformed the data through several phases.

Initially, our objective was to obtain a rough download of the data from the private database, referred to as an "export set." This set included raw JSON text fields and dates represented as strings, providing our team with access to the unprocessed data. The next step involved cleaning the data to create a "cleaned set," where dates were converted to numerical formats, and only the relevant fields were extracted from the JSON structures. We retained unique identifiers in the cleaned set to facilitate debugging and data source tracking. Finally, we prepared an "evaluation set" by dropping unnecessary columns, applying encoding methods, and formatting the data for model training. This phased approach ensured flexibility, allowing us to make adjustments as needed.

The export set comprised 39 columns, including 12 columns of JSON data containing detailed information. Key features of the dataset included employee tenure, timestamped job proficiency ratings, and records of leadership status changes.

2.2 Data cleaning

One of the most challenging aspects of this project was cleaning the data using preprocessors. The data was private and not in the most machine-readable format, so we had to handle it with caution to ensure it was properly prepared for effective model training. The data presented various challenges, from straightforward tasks like converting boolean values to integers, to more complex JSON-formatted data that required parsing. For the JSON data, we carefully extracted the necessary information and created new columns to represent it. We also organized the data into meaningful categories, such as regions and age groups, and converted time information into epoch seconds to maintain accuracy and consistency across the dataset.

2.3 Explanation of Cleaned Data

To achieve better accuracy when testing each model, we utilized a dataset containing 42 features, each carefully chosen to capture essential aspects of the data and ensure accurate predictions. The features were gathered, cleaned, and represented to maintain consistency and relevance. For example, the `businessId`, which uniquely identifies each company, and the city were parsed from JSON format to extract corresponding state information. Additionally, states were categorized into broader regional groups to simplify geographic analysis.

Key decisions included parsing and reorganizing data into meaningful categories to better align with the problem's objectives. Boolean features, such as `hasTraining`, were similarly transformed. Derived features were calculated, such as `worked_months` (the difference between an employee's start and quit dates) and `hired_months_ago` (time elapsed since hiring). For age-related analysis, employees were categorized into three `age_group` categories, with integer indices (`age_group_idx`) assigned for processing efficiency. Furthermore, additional features were introduced to represent specific data indices, making it easier for the model to process. The primary features focused on evaluating performance. For instance, `activeInfractions_points` quantified behavioral infractions, while historical data such as `position_history_length` and `position_history_unique_days` captured employees' work patterns—representing the duration spent in a position and the variety of positions held, respectively. A significant effort went into parsing JSON data for various employee ratings (e.g., `rating@bagger`, `rating@cashCart`, `rating@dishes`). Given the numerous ratings, we grouped them into categories to reduce complexity. These steps were critical in ensuring the dataset was clean, structured, and aligned with the problem's requirements. The primary goal of this project is to focus on six critical features: `foh_pos_loc_idx`, `foh_pos_gbl_idx`, `boh_pos_loc_idx`, `boh_pos_gbl_idx`, `highest_role_loc_idx`, and `highest_role_gbl_idx`. These features represent data derived from `business_Foh_Config_Training`, `business_Boh_Config_Training`, and `businessConfigLeadership`, along with their respective indices. The index features are normalized as lists of position IDs to facilitate better comparisons, ensuring consistency and enabling effective analysis of positional and role-related data across the dataset.

3 Models

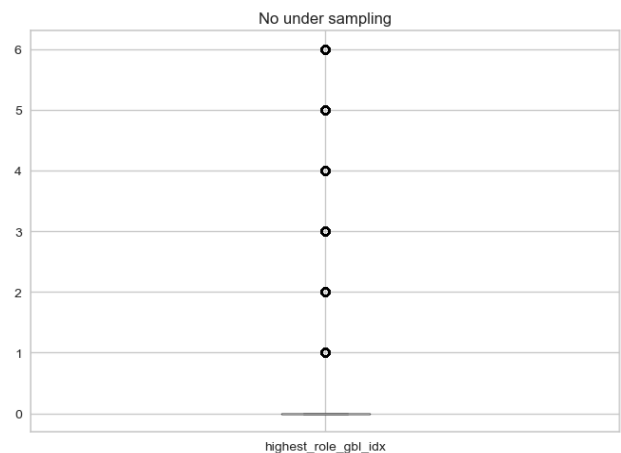
3.1 Understanding the data

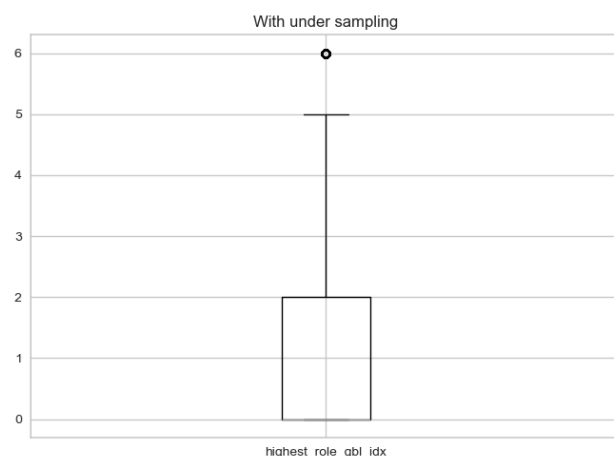
Machine learning models have numerous practical applications, one of which is to interpret data and identify

impactful variables. For this purpose, the selected models must be easy to interpret. Therefore, we have chosen several linear models for data interpretation. In the following analysis, we display results from a test set comprising 25% of the data. We utilize the scikit-learn Perceptron for classification predictions on applicable targets and the scikit-learn MLPRegressor with a single node for continuous predictions on applicable targets. Finally, we employ the statsmodels.api linear regression on the most linearly separable target to determine significant variables based on their coefficients. Although these linear models did not prove to be the most accurate, they are simpler and easier for us to interpret.

Upon closer examination, we realized that the data was significantly skewed, with 87% of the employees being regular team members. We assumed this would pose a problem because machine learning is designed to learn underlying patterns in the data. If the predominant pattern is that most employees are team members, the model will likely predict "team member" consistently. To address this issue, we implemented an under-sampling strategy. Specifically, we reduced the percentage of team members from 87% to 25% by randomly removing team members. Consequently, approximately 25% of the total dataset comprised team members, maintaining the baseline accuracy, as no other role exceeded 25%.

Below are two boxplots showing the results for the highest_role_gbl_idx variable before and after under-sampling.





The test set results of running the Perceptron on all targets are shown below.

No under sampling		With under sampling	
Target	Accuracy (baseline = 0.87)	Target	Accuracy (baseline = 0.25)
boh_pos_gbl_idx	0.95223	boh_pos_gbl_idx	0.741935
boh_pos_gbl_idx	0.939391	boh_pos_gbl_idx	0.711775
boh_pos_loc_idx	0.929864	foh_pos_gbl_idx	0.700729
boh_pos_loc_idx	0.925183	boh_pos_loc_idx	0.699517
highest_role_gbl_idx	0.87525	foh_pos_gbl_idx	0.590875
highest_role_loc_idx	0.859016	highest_role_loc_idx	0.581492
foh_pos_gbl_idx	0.850667	highest_role_loc	0.54103
foh_pos_gbl_idx	0.850584	foh_pos_loc_idx	0.493163
foh_pos_loc_idx	0.835117	foh_pos_loc_idx	0.47023
highest_role_loc	0.827534	boh_pos_loc_idx	0.441691
foh_pos_loc_idx	0.562238	highest_role_gbl_idx	0.249818

The previous results demonstrate that some, but not all, targets improved after under-sampling for classification. The *highest role gbl idx* remained at baseline accuracy for both datasets. However, *foh_pos_loc_idx*, which was 30% below baseline accuracy before under-sampling, improved to 22% above baseline accuracy after under-sampling.

The test set results of running the MLPRegressor with one node on all numerical targets are shown below.

No under sampling		With under sampling	
Target	R ²	Target	R ²
foh_pos_loc_idx	0.707682	foh_pos_loc_idx	0.67071
foh_pos_gbl_idx	-0.000002	boh_pos_gbl_idx	0.000289
highest_role_loc_idx	-0.000017	highest_role_loc_idx	-0.000018
highest_role_gbl_idx	-0.0001	highest_role_gbl_idx	-0.000122
boh_pos_gbl_idx	-0.001743	foh_pos_gbl_idx	-0.096338
boh_pos_loc_idx	-0.12314	boh_pos_loc_idx	-0.424247

Determining the usefulness of under-sampling for regression is challenging, but there is evidence of improved accuracy for *foh_pos_loc_idx*. Before and after

under-sampling, *foh_pos_loc_idx* is the only target achieving a decent R². After under-sampling, it remains difficult to ascertain if this variable has improved, as R² cannot be directly compared to baseline accuracy. However, given a 62% decrease in baseline accuracy, the fact that R² only decreased by 7% indicates improved regression accuracy.

Following these tests, we determined that under-sampling and using *foh_pos_loc_idx* as the target would be optimal for statistical analysis.

Below are the 20 most influential coefficients of linear regression on the *foh_pos_loc_idx* target.

Feature	Coefficient
group_foh	1.721718
age_group_young minor	-1.397282
state_NM	-1.122149
state_RI	1.023545
state_PA	0.941447
rating@bagger	0.83721
state_ID	-0.813985
state_AB	0.790316
age_group_minor	-0.750215
state_NC	0.678627
state_MS	-0.658827
state_WI	-0.620984
state_ND	0.60516
age_group_idx	-0.570982
hasTraining	0.544812
state_CT	-0.531043
region_SOUTHWEST	-0.525831
hasProfileUrl	0.514195
state_ZZ	-0.504122
state_TX	0.284538

This analysis aims to enhance our understanding of the data. We now know that an individual's bagger rating is more influential in prediction than their minor status. Although other machine learning models may not follow this logic, understanding the data is beneficial. It not only helps us learn what the machines might be learning but also aids in identifying mistakes.

All previous results in the "Understanding the Data" section were gathered after identifying a mistake in the machine learning process. The mistake was that the *group_leader* feature was not removed from the dataset. Initially, we ran the statistical analysis on the most effective regressor target before dropping *group_leader*, which was *highest_role_loc_idx*.

Below are the 20 most influential coefficients of linear regression on the *highest_role_loc_idx* target before removing *group_leader*:

Feature	Coefficient
state_ZZ	5.244448
group_leader	3.93843
hasUserId	0.831159
hasProfileUrl	0.751354
hasPin	0.704295
region_OTHER	0.584759
state_NM	0.567231
state_ME	0.505491
age_group_adult	0.394765
state_WA	0.377989
state_MO	0.366554
state_RI	0.342489
state_AL	0.33622
foh_pos_loc_id_is_nan	0.2292
state_MN	0.226354
boh_pos_loc_id_is_nan	0.214937
business_num_roles	0.211561
state_OR	0.164997
rating@screens	0.119616
foh_pos_gbl_id_is_nan	0.09953

The *group_leader* feature is evidently more influential than most others because it leaks data. It should not be included in the features, as being a group leader indicates a specific role.

3.2 Multi-Layer Perceptron

Initially, we trained the Multilayer Perceptron (MLP) model on a dataset that included several target columns. The results showed exceptionally high accuracy metrics:

- Train Accuracy for *foh_pos_gbl_idx*: 0.9997
- Test Accuracy for *foh_pos_gbl_idx*: 0.9899
- Train Accuracy for *boh_pos_gbl_idx*: 0.9997
- Test Accuracy for *boh_pos_gbl_idx*: 0.9958
- Train Accuracy for *highest_role_gbl_idx*: 0.9997
- Test Accuracy for *highest_role_gbl_idx*: 0.9974
- Overall Train Accuracy: 0.9991
- Overall Test Accuracy: 0.9832

While these results looked impressive at first glance, further analysis revealed that the dataset included columns such as *highest_role_loc_idx*, *boh_pos_loc_idx*, and *foh_pos_loc_idx*, which were highly correlated with the target labels. This created information leakage, artificially inflating the accuracy metrics. To address this, we removed these columns, along with *group_leader*, to ensure that the model's performance reflected genuine predictive capabilities.

Once these columns were removed, the MLP model was retrained. The updated results showed a noticeable drop in accuracy, which was expected given the removal of leaked information:

- Train Accuracy for *foh_pos_gbl_idx*: 0.9951
- Test Accuracy for *foh_pos_gbl_idx*: 0.9867
- Train Accuracy for *boh_pos_gbl_idx*: 0.9990
- Test Accuracy for *boh_pos_gbl_idx*: 0.9940
- Train Accuracy for *highest_role_gbl_idx*: 0.9067
- Test Accuracy for *highest_role_gbl_idx*: 0.8883
- Overall Train Accuracy: 0.9011
- Overall Test Accuracy: 0.8708

These updated results provide a more realistic picture of the model's capabilities. The removal of information leakage ensured that predictions were based on meaningful patterns in the data rather than spurious correlations.

To contextualize the re-trained model's performance, we compared its accuracy against baseline metrics for each target:

- Baseline accuracy for *highest_role_gbl_idx*: 87.34%
- Baseline accuracy for *foh_pos_gbl_idx*: 80.31%
- Baseline accuracy for *boh_pos_gbl_idx*: 90.63%

The updated model showed substantial improvements over baseline accuracy for *foh_pos_gbl_idx* and *boh_pos_gbl_idx*, achieving test accuracies of 98.67% and 99.40%, respectively. For *highest_role_gbl_idx*, the test accuracy of 88.83% was only slightly higher than the baseline, indicating potential areas for further improvement.

3.3 K-Nearest Neighbor (KNN)

We began by evaluating different KNN configurations and comparing them against the baseline accuracy of 87.34%.

The initial approach involved using basic KNN classification with Manhattan distance ($p=1$). The feature *boh_pos_gbl_idx* provided the highest accuracy of 96.84%, though it also had the highest error (47,010). In contrast, using Euclidean distance ($p=2$), *foh_pos_gbl_idx*

achieved an accuracy of 93.92% but had a significantly higher error (5,464,707). The high accuracy was likely due to the chosen distance metric (Manhattan), and the high error (5,464,707) suggests that the model might be overfitting.

p	Feature	Avg Test Accuracy	Predicted Accuracy	SSE
1	foh_pos_gbl_idx	0.95839	0.95839	34549
1	boh_pos_gbl_idx	0.968416	0.968416	47010
1	highest_role_gbl_idx	0.935977	0.935977	13101
2	foh_pos_gbl_idx	0.939228	0.939228	5464707
2	boh_pos_gbl_idx	0.869833	0.869833	1640959
2	highest_role_gbl_idx	0.874795	0.874795	44561

Next, we approached the problem without normalization or distance weighting. We compared results for $N_{\text{neighbors}} = 3$ and 5. This approach did not show better accuracy than the first approach, and both results showed similar average test accuracy percentages: 94.05% and 93.92% under *foh_pos_gbl_idx*.

N_neighbors	Feature	Avg Test Accuracy	Predicted Accuracy	SSE
3	foh_pos_gbl_idx	0.940563	0.940563	3969465
3	boh_pos_gbl_idx	0.862408	0.862408	1581960
3	highest_role_gbl_idx	0.871647	0.871647	43696
5	foh_pos_gbl_idx	0.939228	0.939228	5464707
5	boh_pos_gbl_idx	0.869833	0.869833	1640959
5	highest_role_gbl_idx	0.874795	0.874795	44561

We then introduced Z-score normalization, which led to a significant improvement in accuracy. With $N_{\text{neighbors}} = 5$, the feature *boh_pos_gbl_idx* achieved a high accuracy of 97.92% with a relatively low error (33,150). The feature *highest_role_gbl_idx* had the lowest accuracy at 95.18%, but it showed the smallest error (8,816). Additionally, min-max normalization showed that *foh_pos_gbl_idx* achieved 97.12% accuracy with a lower error of 9,979. With normalization techniques such as Z-score normalization or min-max normalization, the model performed excellently.

Normalize	Feature	Avg Test Accuracy	Predicted Accuracy	SSE
Z-score	foh_pos_gbl_idx	0.962599	0.962599	45683
Z-score	boh_pos_gbl_idx	0.979195	0.979195	33150
Z-score	highest_role_gbl_idx	0.95182	0.95182	8816
min-max	foh_pos_gbl_idx	0.974884	0.971154	9979
min-max	boh_pos_gbl_idx	0.974884	0.974884	21337
min-max	highest_role_gbl_idx	0.92513	0.92513	8265

In the end, applying a combination of normalization and distance weighting led to the highest accuracy results. The model achieved a perfect training score of 1.0 for all features when normalized and distance weighting were applied, and high test accuracies of 98.83% for *boh_pos_gbl_idx*, 96.92% for *foh_pos_gbl_idx*, and 93.08% for *highest_role_gbl_idx*. The concern is that since the test accuracy for *highest_role_gbl_idx* is lower than others, it might be overfitting. The Mean Absolute Error (MAE) was very low, especially for the *boh_pos_gbl_idx normalized distance weighting* model, which had a test MAE of 0.024.

Feature	Train Score	Test Score	Train MAE	Test MAE
foh_pos_gbl_idx (No Normalization, No Distance Weighting)	0.916371	0.857515	0.1979417	0.273323
foh_pos_gbl_idx (Normalized, No Distance Weighting)	0.978689	0.967449	0.0814906	0.105831
foh_pos_gbl_idx (Normalized, Distance Weighting)	1	0.969188	0	0.101205
boh_pos_gbl_idx (No Normalization, No Distance Weighting)	0.881939	0.815131	0.128469	0.170866
boh_pos_gbl_idx (Normalized, No Distance Weighting)	0.991715	0.987678	0.019909	0.025602
boh_pos_gbl_idx (Normalized, Distance Weighting)	1	0.988327	0	0.024388
highest_role_gbl_idx (No Normalization, No Distance Weighting)	0.89382	0.81846	0.094299	0.127286
highest_role_gbl_idx (Normalized, No Distance Weighting)	0.951999	0.926121	0.0703607	0.08892
highest_role_gbl_idx (Normalized, Distance Weighting)	1	0.930797	0	0.083921

The iterative steps we took—starting with basic KNN, then adding normalization, and finally applying distance weighting—led to significant improvements in accuracy. The final model, which included both normalization and distance weighting, achieved test accuracies as high as 98.83%, significantly surpassing the baseline accuracy of 87.34%.

3.4 Decision Tree

We used Decision Tree models to predict the output targets of the dataset. We applied the same transformations to the cleaned data set as were used in the previous models, namely: we dropped identification and string columns, created new columns to signal the presence of missing values as needed, and one-hot encoded the *region* field. Decision tree models are only able to predict one target column at a time; we set up models to individually train the models on each of the target columns independently and report the results.

The initial results were starkly different between the `DecisionTreeClassifier` and the `DecisionTreeRegressor`.

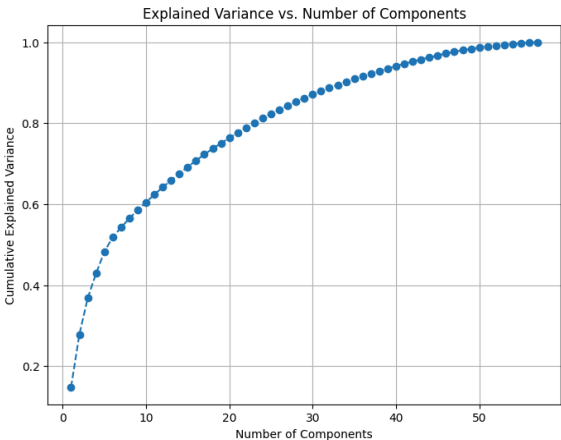
While the classifier model was able to score 85.0% on the *highest_role_gbl_idx* target, the regressor model scored an abysmal 14.5%. This suggests that our problem is fundamentally a classification problem rather than a regression problem. This surprised us because we had expected to find substantial correlation between the time an employee had been working and their level of promotion. Both models showed significant gaps in accuracy between the training sets and the test sets. The classifier reported a gap of 14.2% and the regressor reported a gap of 80.2%. This accuracy gap suggests that the models were experiencing severe overfit issues which would need to be resolved.

We explored diverse ranges of values of the Decision Tree hyperparameters in order to address overfit and increase testing accuracy. We adjusted each of the following parameters with values that spanned the entire range of allowable values: *ccp_alpha*, *min_samples_leaf*, *min_samples_split*, *min_impurity_decrease*, *max_depth*, *max_leaf_nodes*, *max_features*. Each of these independently improved the test accuracy by ~3% in the classifier models and ~27% in the regressor models. Even with the sharp improvement shown by the regressor, the regressor's maximum accuracy of 43.9% does not qualify it for serious consideration.

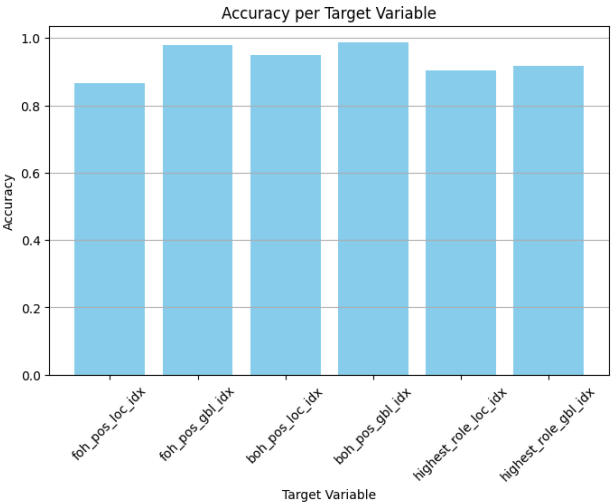
3.5 Principal Component Analysis (PCA)

We applied Principal Component Analysis (PCA) to reduce the dimensionality of our dataset and improve computational efficiency. Initially, the dataset had a large number of features, which we suspected might introduce noise or redundant information into the model. To tackle this, we focused on the cumulative explained variance to determine the optimal number of components to retain.

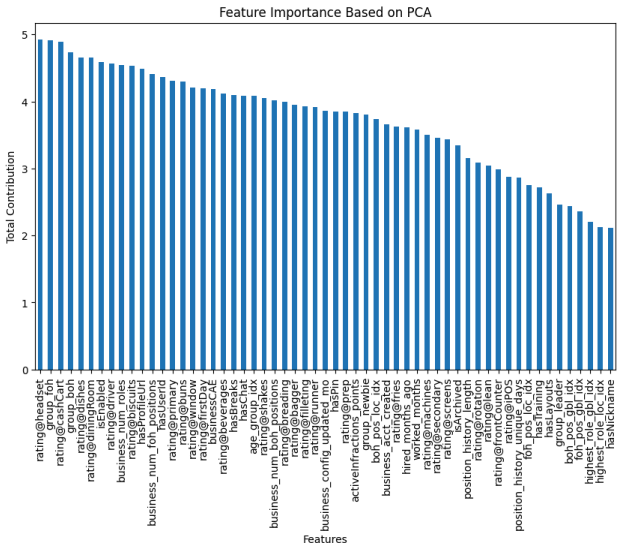
From our analysis, we found that the first 40 principal components captured a significant portion of the variance in the data. This was a good balance between reducing the number of features and keeping the most meaningful information. The cumulative explained variance plot clearly showed diminishing returns after 40 components, so we decided to use this as our cutoff.



After applying PCA with 40 components, we tested our model's performance on multiple targets. The accuracy for *foh_pos_loc_idx* came out to 86.73%, while *foh_pos_gbl_idx* performed exceptionally well with an accuracy of 98.07%. Other targets, such as *boh_pos_loc_idx* and *highest_role_gbl_idx*, also showed strong performance with accuracies of 94.94% and 91.73%, respectively. Overall, the average test accuracy across all targets was 93.41%. The total sum of squared errors (SSE) across all targets was 193,950, showing that the PCA transformation retained enough information to maintain robust predictions.

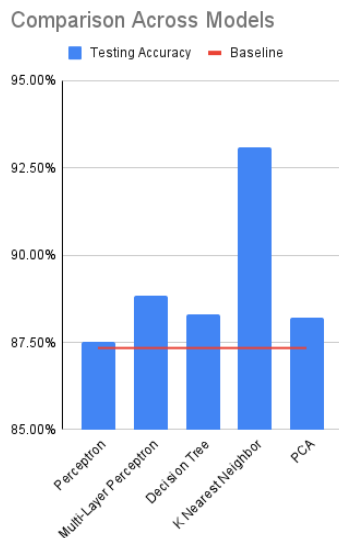


Looking deeper into the principal components, features like *rating@headset*, *group_foh*, and *rating@cashCart* stood out as significant contributors to the variance in the dataset. These results reassured us that PCA not only improved model efficiency but also helped highlight the most impactful features in the dataset.



4 Comparison of Selected Models

There are several target variables that we investigated with our models, but the most prominent target is the *highest_role_gbl_idx*. The best training accuracy results for each of the models are visualized below. Most of the models beat the baseline accuracy by only slim amounts. The K-Nearest Neighbor model significantly outperformed all the others. We believe this is due to its ability to compare individual employees to the other records within their own restaurant. After discovering that the KNN model outperformed the next best model by 4.25%, our understanding of the problem shifted significantly. It became clear that the issue is more about comparing employees within the same business contexts and situations. The patterns are more complex than initially anticipated.



5 Conclusion

What began as an expectation of a simple and straightforward prediction model with a linear relationship evolved through testing and experimentation. We realized that the problem was not linear; instead it appears to rely heavily on the local decision and local consistency within small groups of employees sharing leadership and regulatory environment. With such a diversity of ways to run and manage a restaurant, and with diverse levels of engagement with the source software platform, generalizations to other businesses appear to be inefficient and unpredictable.

6 Future Work

Given more time, there are several future developments we could pursue to improve our understanding of the data and possibly enhance our accuracy. First, we would run

and interpret clustering, and second, we would try Naive Bayes.

Clustering has a Big O notation of $O(n^3)$, indicating that it requires a significant amount of time to compute. We began the process of interpreting clusters developed by k-means, but due to the extended computation time, we decided to leave this for a future project.

Naive Bayes would be interesting to explore because, despite its simplicity, it could potentially detect patterns in the data that other models might miss.